

INF. II – mod.2 // Comp Intell

E9: evolutionary algorithms

Marco S. Nobile, Ph.D.

Bachelor's Degree in Engineering Physics

Ca' Foscari University of Venice

A.Y. 2025-2026

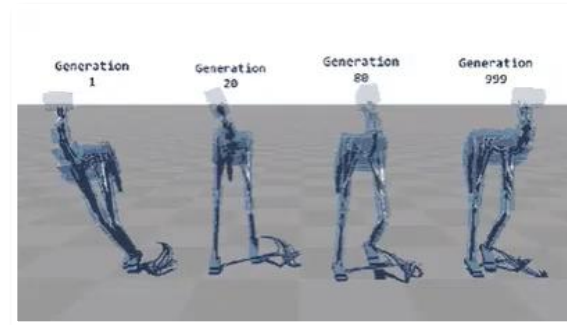


Università
Ca' Foscari
Venezia

The power of evolution



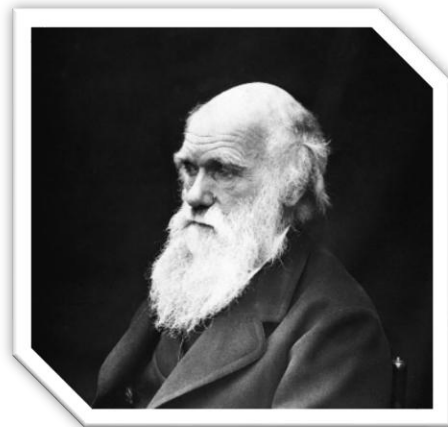
The Evolution of Bacteria on a "Mega-Plate" Petri Dish (Kishony Lab)
<https://www.youtube.com/watch?v=plVk4NVIUh8>



Exploiting evolution for optimization problems



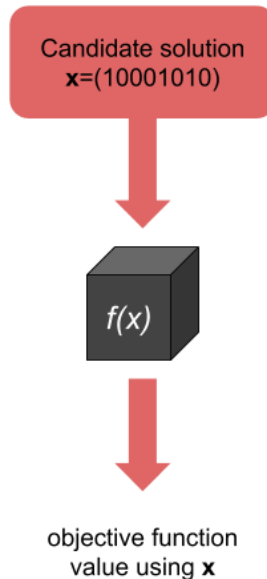
- The idea is to create a set of **random candidate solutions** to a given optimization problem and simulate Darwinian processes to **“evolve” the population towards optimal solutions**
- Optimal with respect to the **selection pressure induced by the objective function**
- **What is a candidate solution?**



Candidate solutions

In the simplest form of GAs, a **candidate solution** is encoded as a **fixed-length vector** created by composing symbols taken from a **finite alphabet**, for example:

- Strings of bits
- Lists of characters
- Arrays of letters, numbers...
- One vector $\mathbf{x}$ **encodes a feasible solution** to the problem and its “quality” can be **evaluated** by means of an **objective function** f
- In the field of evolutionary computation the candidate solutions take the name of **individuals**
- A set of individuals is named a **population**





A straightforward example: One Max

- **The One Max problem** is a simple toy optimization problem consisting in **maximizing the number of ones** in a bitstring of length M
 - Candidate solutions are represented by **strings of M bits** (i.e., made of 0s and 1s)
 - For instance, in the case $M = 8$, one solution could be $\mathbf{x} = (10101100)$
- One **possible fitness function** for the One Max problem could be defined as:

$$f(\mathbf{x}) = \sum_{i=1}^M x_i$$

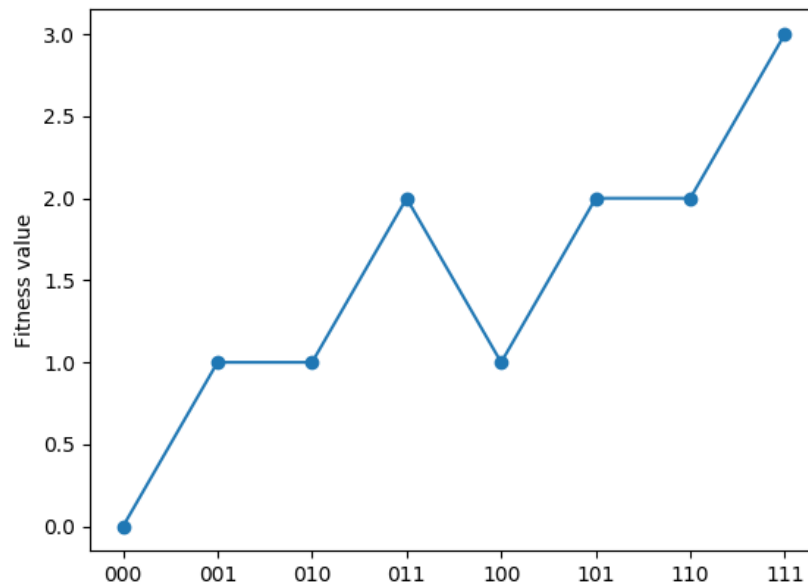
where x_i is the value of the bit in position i

- One Max is a “toy” problem, but helps you to grasp the concepts of candidate solutions **representation** and **fitness evaluation**

The fitness landscape of the One Max problem



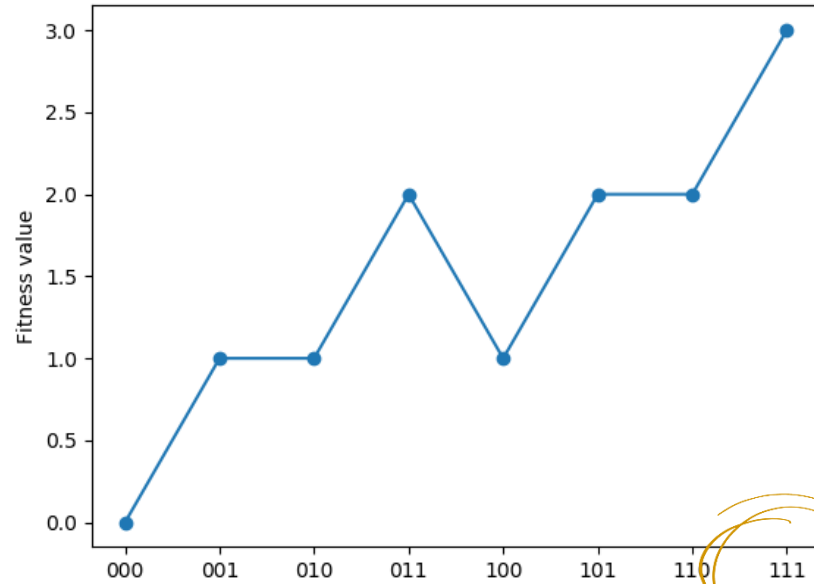
- One Max problem with $M = 3$



The fitness landscape of the One Max problem



- One Max problem with $M = 3$

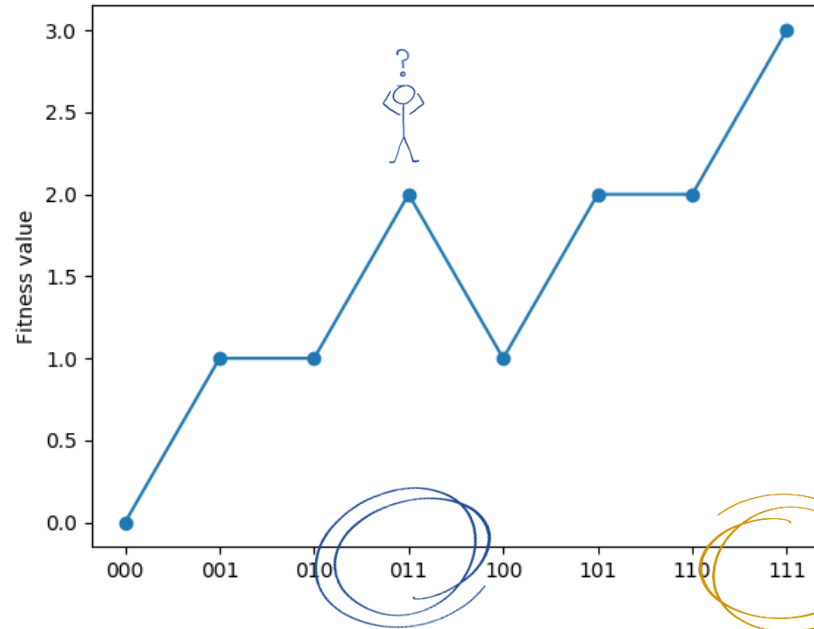


Optimal solution

The fitness landscape of the One Max problem



- One Max problem with $M = 3$



Optimal solution



Genetic Algorithms

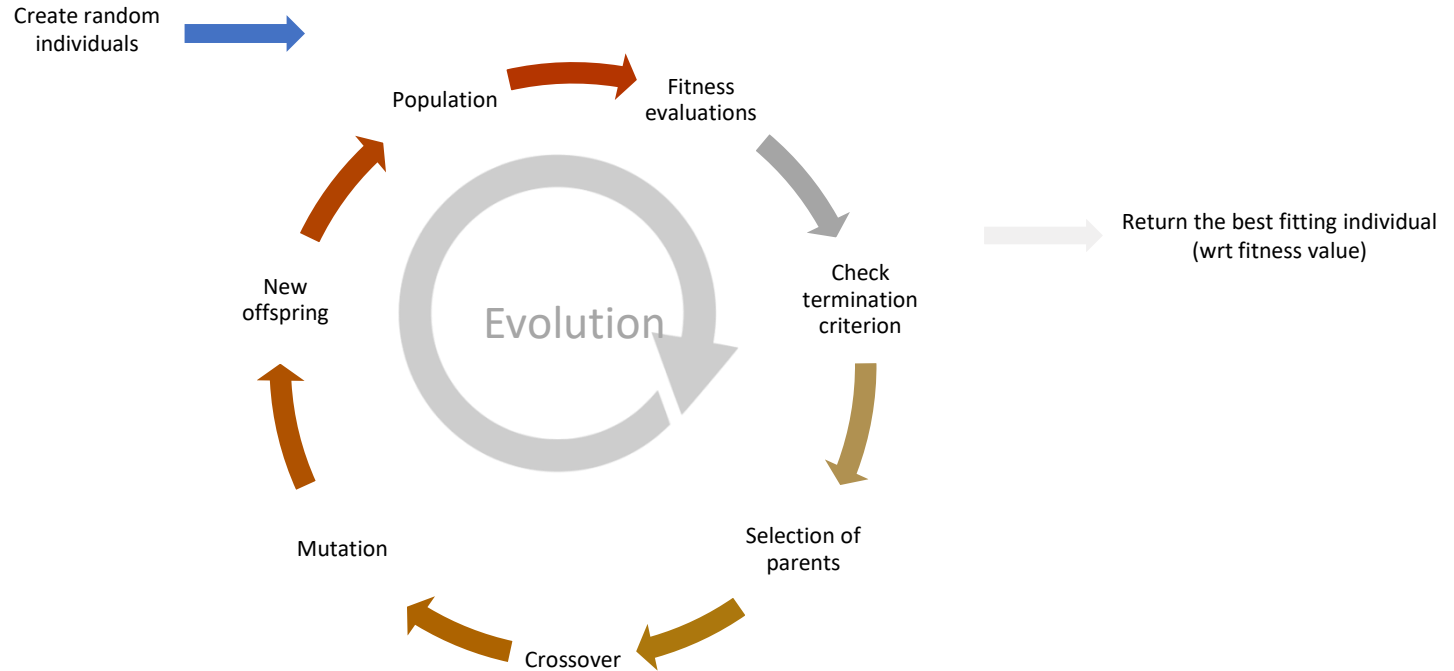
- Introduced by **John Holland** in the 70s
[*Adaptation in Natural and Artificial Systems*, MIT Press, 1975]
- Based on the concepts of **evolution and natural selection**
- A set of randomly generated candidate solutions **evolves** iteratively and **converges to the optimal solution** of a given problem
- GAs are **widely used** in basically **every discipline**, and they can be very effective in the case of complex problems on large search spaces
- GAs can work with both discrete and real-valued candidate solutions (more about that later)

Basic functioning of a standard GA

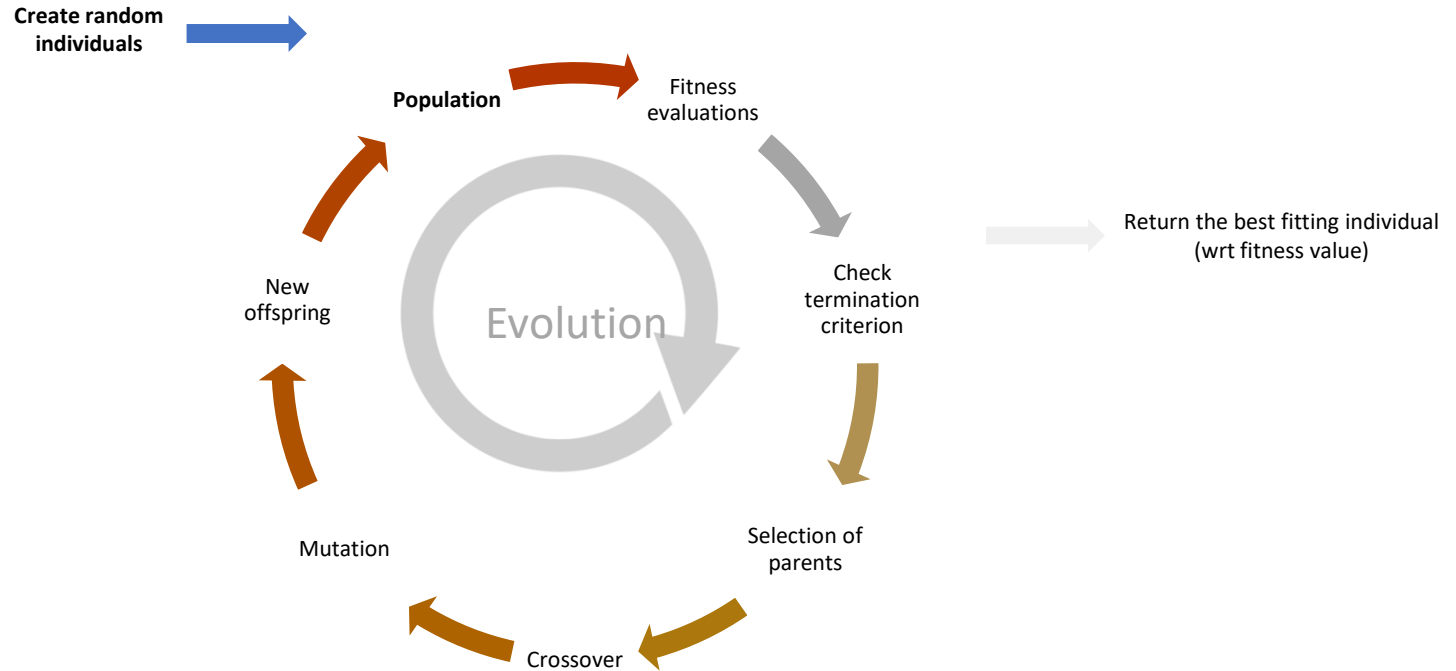


1. A population of random N individuals is **created**
2. The **fitness** value of all N individuals is calculated
3. *Survival of the fittest*: a **selection mechanisms** is used to choose **pairs** of individuals with a probability (to some extent) proportional to their fitness values
4. Each pair of selected individuals (the **parents**) undergoes a **genetic crossover**: their chromosomes are **randomly exchanged** to produce new individuals (the **offspring**)
5. The offspring undergo **genetic mutation**: some symbols of the individuals are randomly changed (e.g., bit flips)
6. When N offsprings are created, they **replace the previous population**
7. If the **termination criterion** is met the algorithm returns the **best fitting individual** as solution; else, **perform a new generation** by iterating the process from step #2

GAs iterative process



GAs iterative process





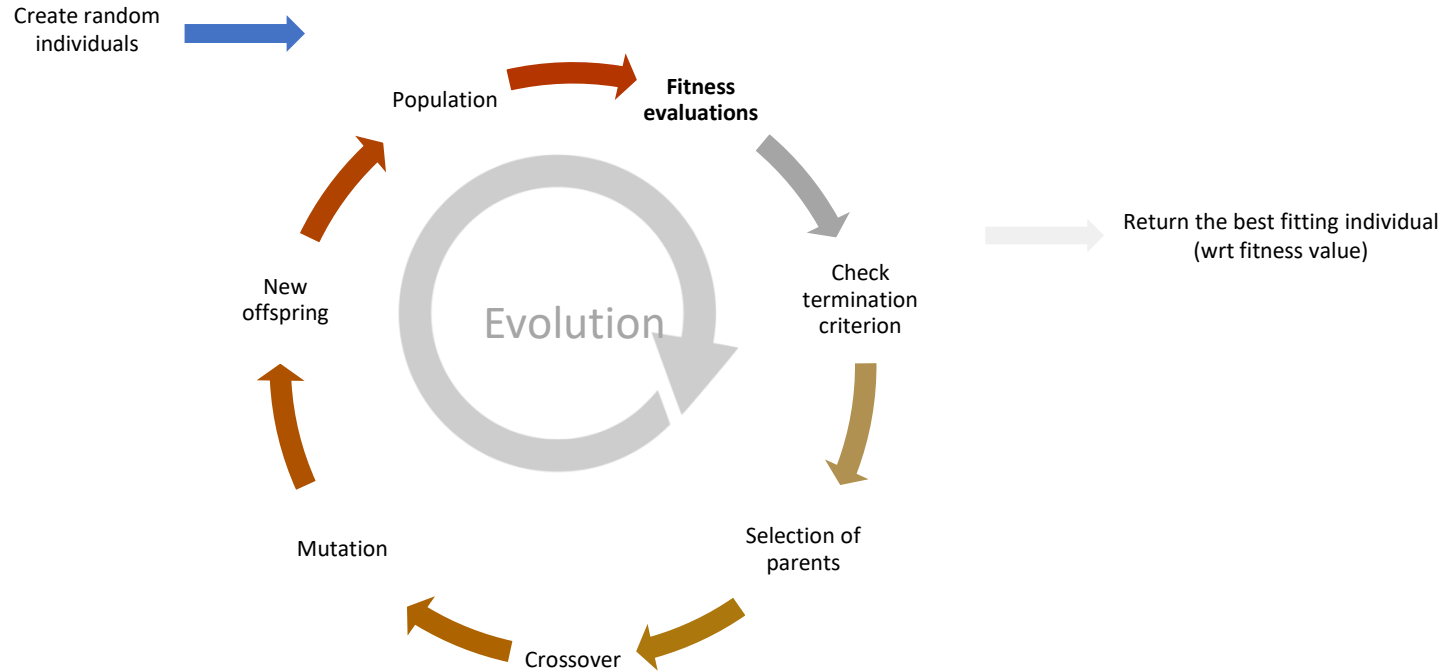
Creating the initial population

- Randomly **create a population** of N individuals
 - For instance, random binary strings in the case of the One Max problem (see example on the right)
- A **uniform distribution** is generally used to choose the symbols of the individuals, unless some *a priori* information is available
 - Pro of uniform: does not introduce any biases in the evolutionary process
 - Con of uniform: might take longer time (i.e., more generations) to converge to optimal solutions

Generating 10 individuals of length 8:

```
x0: 01101111
x1: 11100100
x2: 00010110
x3: 00001000
x4: 10110110
x5: 01011111
x6: 01011110
x7: 10011010
x8: 10000011
x9: 00011010
```

GAs iterative process



Fitness evaluations

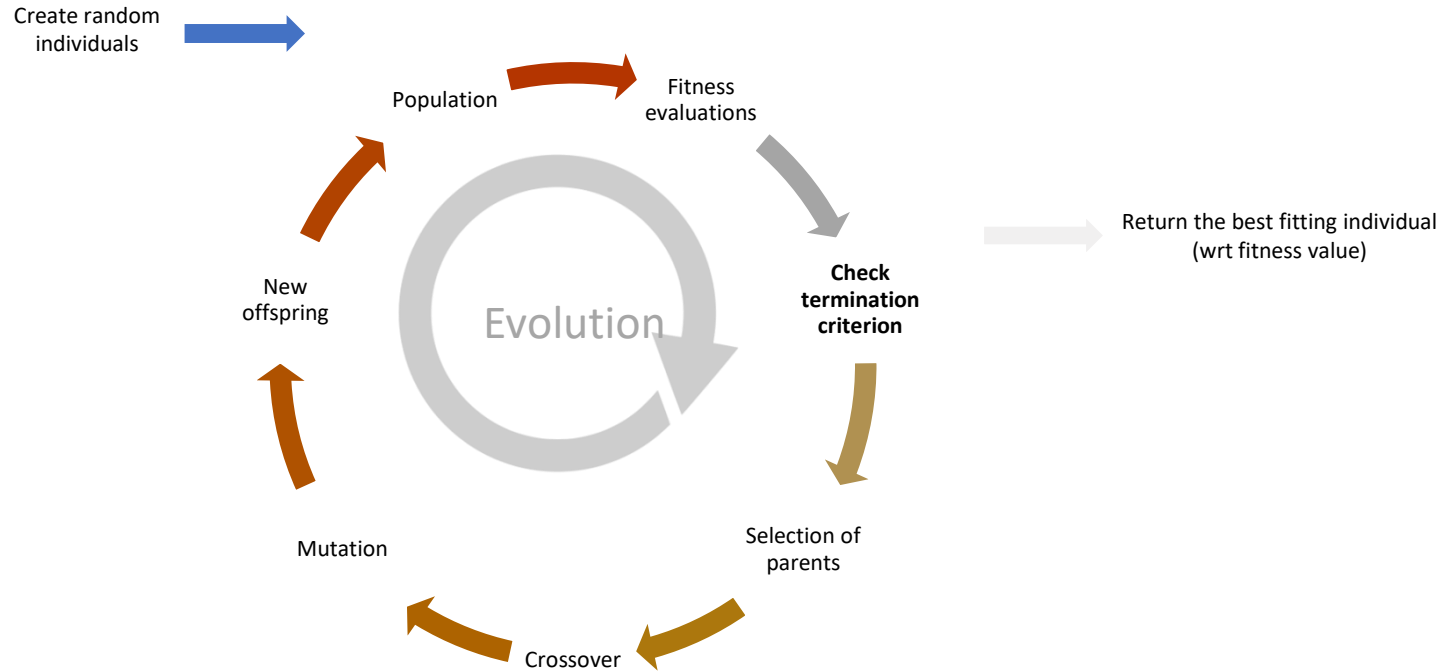


- For each individual in the population we must **calculate the fitness value**
 - In the case of One Max: $f(\mathbf{x}) = \sum_{i=1}^M x_i$
 - That is, we count the number of 1s in the representation
 - In real world scenarios it might be the result of a simulation, the precision of a model, etc.

Evaluating the population:

```
f(x0=01101111): 6
f(x1=11100100): 4
f(x2=00010110): 3
f(x3=00001000): 1
f(x4=10110110): 5
f(x5=01011111): 6
f(x6=01011110): 5
f(x7=10011010): 4
f(x8=10000011): 3
f(x9=00011010): 3
```

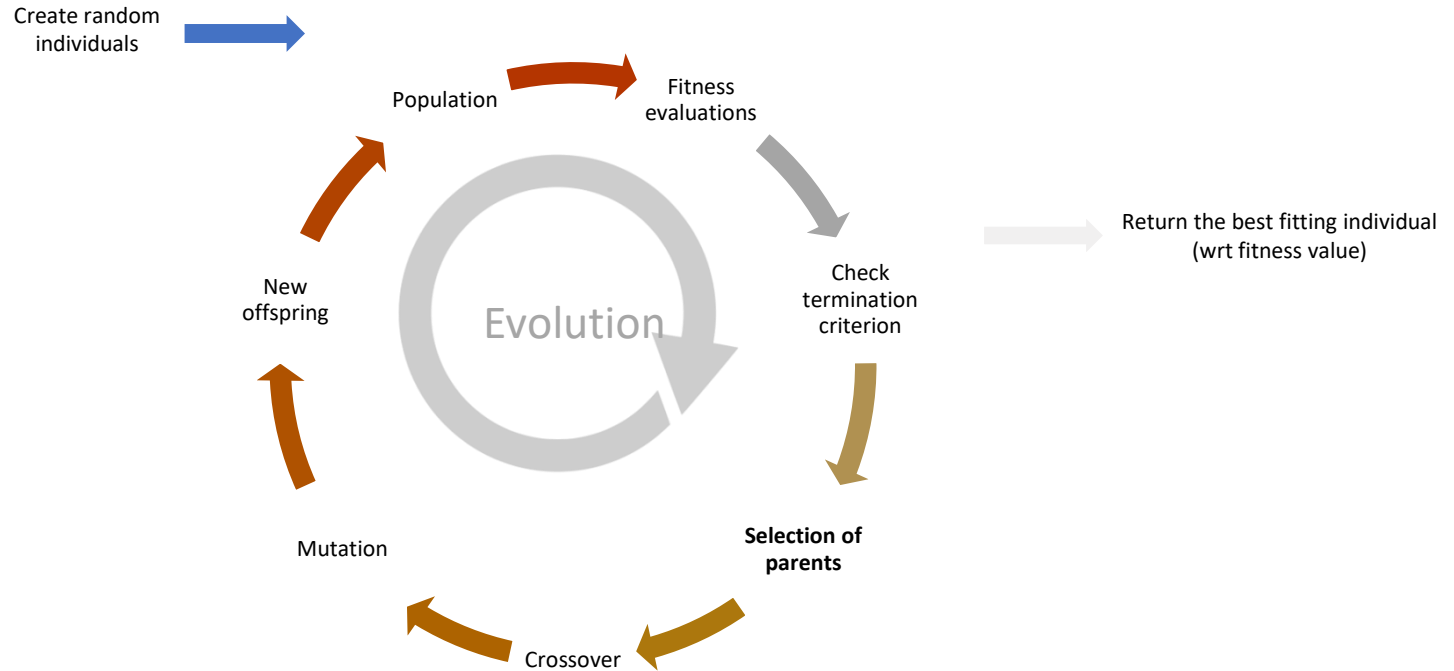
GAs iterative process



Termination criterion

- For the One Max problem, the optimal value is known: it is indeed equal to the maximum number of 1s that we can have in a string of length M
- Hence, since we are considering the One Max problem with $M = 8$, one possible termination criterion is: *"stop when an individual reaches fitness 8"*
- Unfortunately, **the optimal fitness value is unknown** in many real-world scenarios
- In the latter case the GA could use different halting criteria, e.g.:
 - Terminate as soon as the fitness value reaches a user-defined **threshold**
 - Terminate after a **fixed amount of generations**
 - Terminate when the population is too **homogeneous** (loss of diversity)...

GAs iterative process



Selection of parents

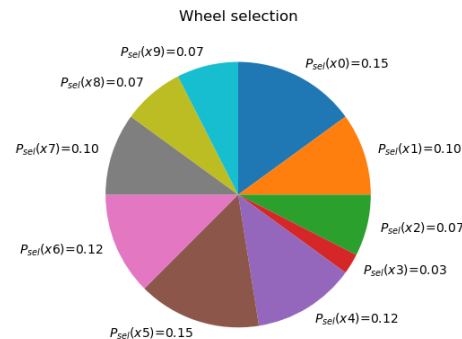


- Selection implements the “survival of the fittest”
- **Multiple selection mechanisms exist** (see, e.g., [Sivaraj and Ravichandran, IJEST 2011])
- They introduce **different selection pressures** on the population, e.g., bad fitting individuals could be quickly removed from the population, which leads to a **loss of diversity** and affects GA’s exploration capability
- Important: there is no free lunch, the optimal selection method is **problem dependent**
- We will see the most used selection methods: **roulette wheel**, **ranking**, and **tournament**

Selection methods: Roulette wheel

- The **probability** of selecting individual $\mathbf{x}_i$ is **directly proportional to its fitness value**:

$$P_{sel}(\mathbf{x}_i) = \frac{f(\mathbf{x}_i)}{\sum_{n=1}^N f(\mathbf{x}_n)}$$



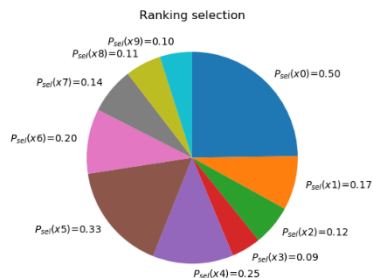
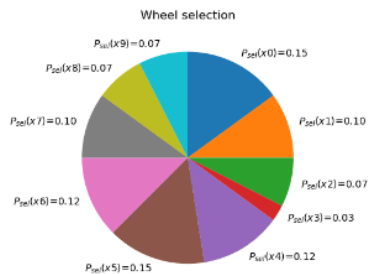
```
f(x0=01101111) : 6
f(x1=11100100) : 4
f(x2=00010110) : 3
f(x3=00001000) : 1
f(x4=10110110) : 5
f(x5=01011111) : 6
f(x6=01011110) : 5
f(x7=10011010) : 4
f(x8=10000011) : 3
f(x9=00011010) : 3
```

- Pro: simple to implement, generally effective
- Con: individuals with low fitness have very little chance of being selected

Selection methods: Ranking

- Rank the solutions according to their quality (i.e., fitness value)
- The **probability** of selecting individual $\mathbf{x}_i$ is **proportional to its ranking** r_i :

$$P_{sel}(\mathbf{x}_i) = \frac{1}{r_i + 1}$$

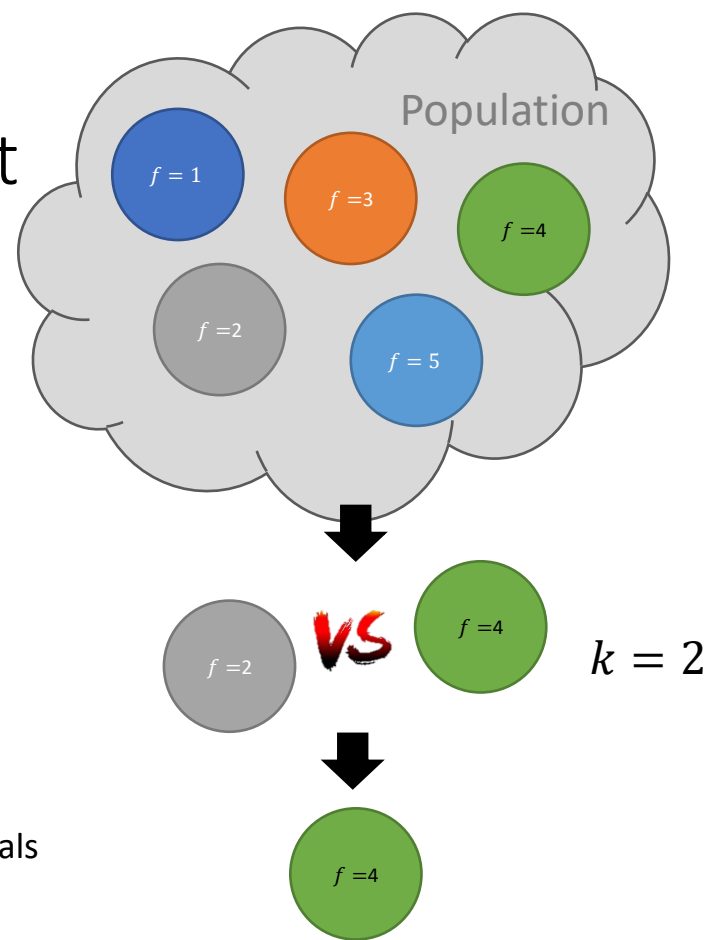


```
f(x0=01101111): 6
f(x1=11100100): 4
f(x2=00010110): 3
f(x3=00001000): 1
f(x4=10110110): 5
f(x5=01011111): 6
f(x6=01011110): 5
f(x7=10011010): 4
f(x8=10000011): 3
f(x9=00011010): 3
```

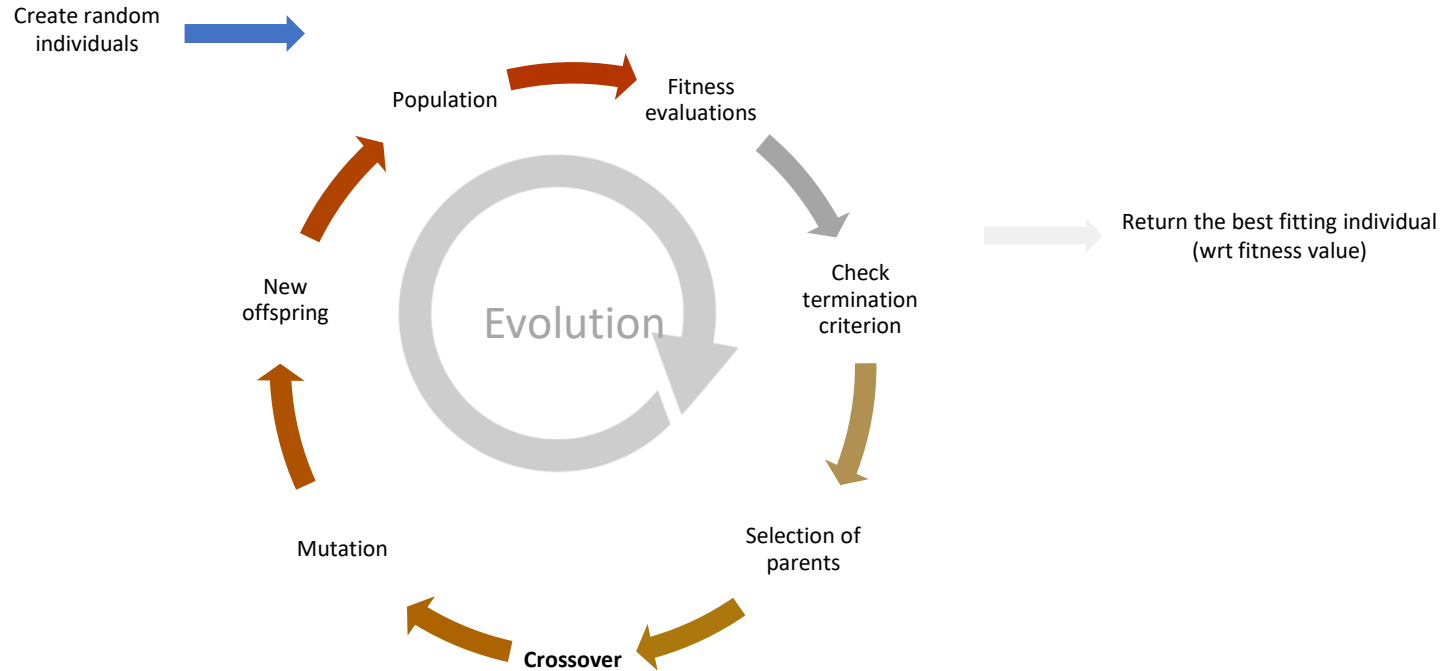
- Pro: low fitness individuals have a higher chance of being selected
- Con: can affect the convergence, the selection of individuals is less dependent on fitness values

Selection methods: Tournament

- $k \ll N$ unique individuals are chosen from the population for a “tournament”
 - The best individual deterministically wins the tournament and **is selected**
 - N separate tournaments are performed
- Existing alternatives: stochastic tournament, individuals chosen with replacement
- Pro: **all individuals have chances** of being selected
- Pro: **selection pressure can be controlled** by the k hyper-parameter (e.g., using large values of k , weak individuals have a smaller chance to be selected)



GAs iterative process

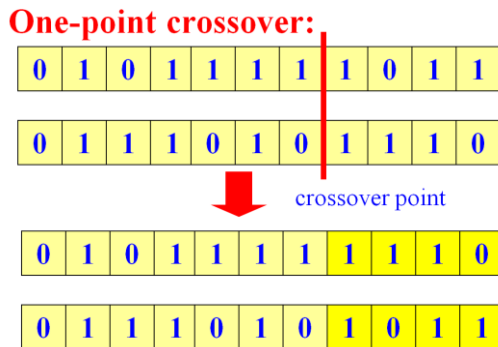


Crossover

- Once the N parents are selected, it is time for mating (=crossover)
 - For each pair of parents, extract a **random number** $r \in [0,1]$
 - If $r < P_c$ then the parents undergo crossover
 - P_c is the **crossover probability** (generally ≈ 0.9)
 - If the parents do not undergo crossover, their offspring directly proceed to random mutation
- Crossover consists in **exchanging a part of the chromosomes** between two individuals
 - The idea/intuition is that two sub-optimal individuals can contain some good “patterns” that, once they are combined, might yield **excellent offspring** (more about this later)
 - Multiple crossover methods exist; we will focus now on the three most widespread methods named **single-point crossover** (SPX), **uniform crossover** (UX), and **partially-matched crossover** (PMX)
 - Mating between **multiple individuals** is also possible (see, e.g., [Eiben *et al.*, ECAL 1995])

Single-point crossover

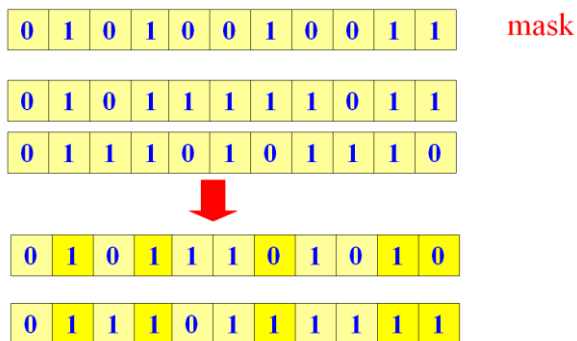
- Also named “One-point crossover”
 - Select a **random crossover point** (see the red line)
 - The two parents are “**cut**” in that **point** and one half is **exchanged between the parents**
 - This process yields **two offspring solutions** proceeding to **random mutation**



- **n-point crossover:** we use multiple “cut” points

Uniform crossover

- Randomly generate a **bit-mask**
- Iterate on the bit-mask and
 - If the i -th bit is 0, copy the bit of parent1 to offspring1 and the bit of parent2 to offspring2
 - If the bit of the bit-mask in position i is **equal to 1** then **exchange the bits** of the parents in position i and apply the previous step



- Pro: uniform crossover prevents any **positional biases**
- Con: **computationally intensive**, a lot of pseudo-random numbers to be generated

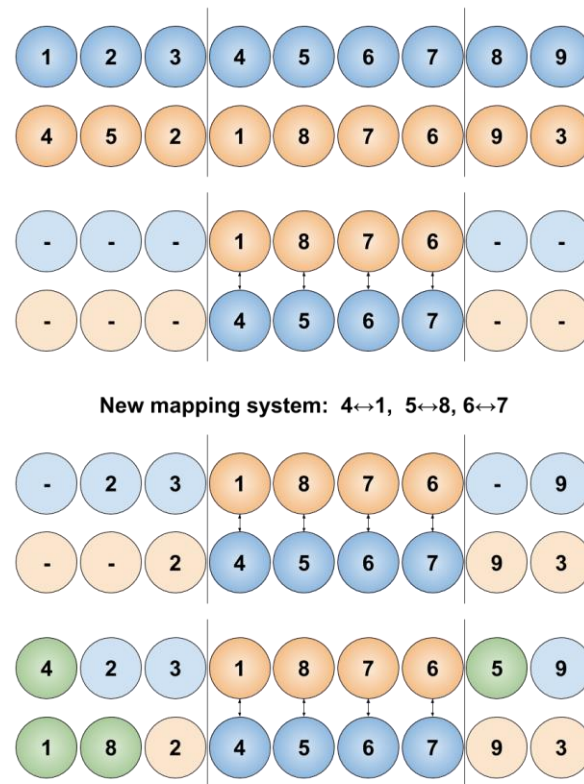
Partially matched crossover



Special type of crossover **preserving relative order**

- For instance PMX (partially-matched crossover), proposed by Goldberg in 1985
- Useful when candidate solutions must preserve some constraints, e.g., represent a **permutation** (can you think of some application?)
- Closed operator: the offspring are **still valid permutations** (without the need for fixing steps)

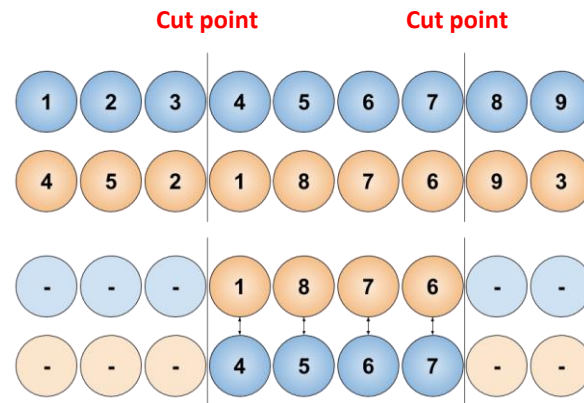
Several alternative, order-preserving crossover operators exist – see [Rutkowski *et al.*, LNAI 2004]



Partially matched crossover

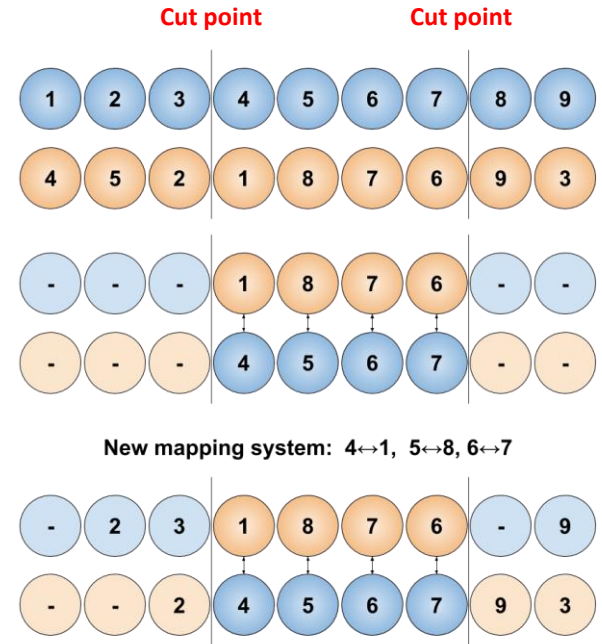


- We have two individuals to be crossed over that are permutations
- We define **two cut points**
- We align the values in the middle to **create a mapping**
(e.g., the 1 is aligned to 4, now 1 maps to four and vice versa)



New mapping system: $4 \leftrightarrow 1$, $5 \leftrightarrow 8$, $6 \leftrightarrow 7$

Partially matched crossover



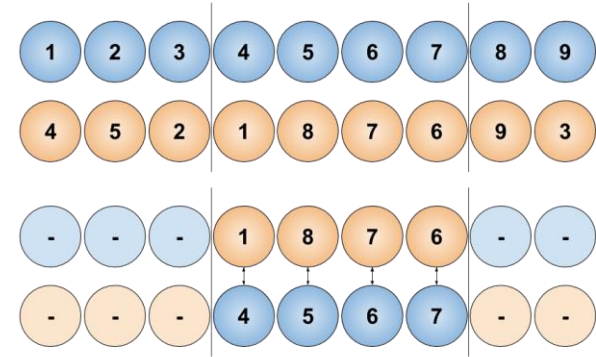
- We now copy **those values that are not part of the mapping** to the new individual **as they were** in the parents

Partially matched crossover

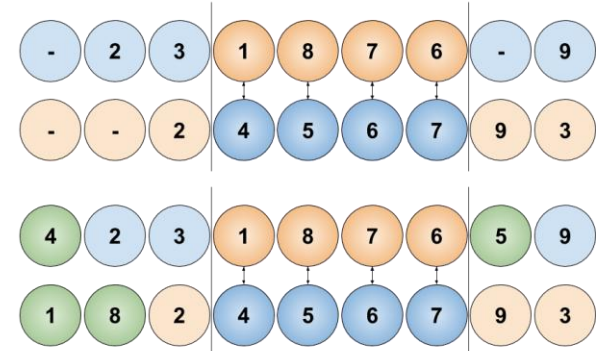


Cut point

Cut point

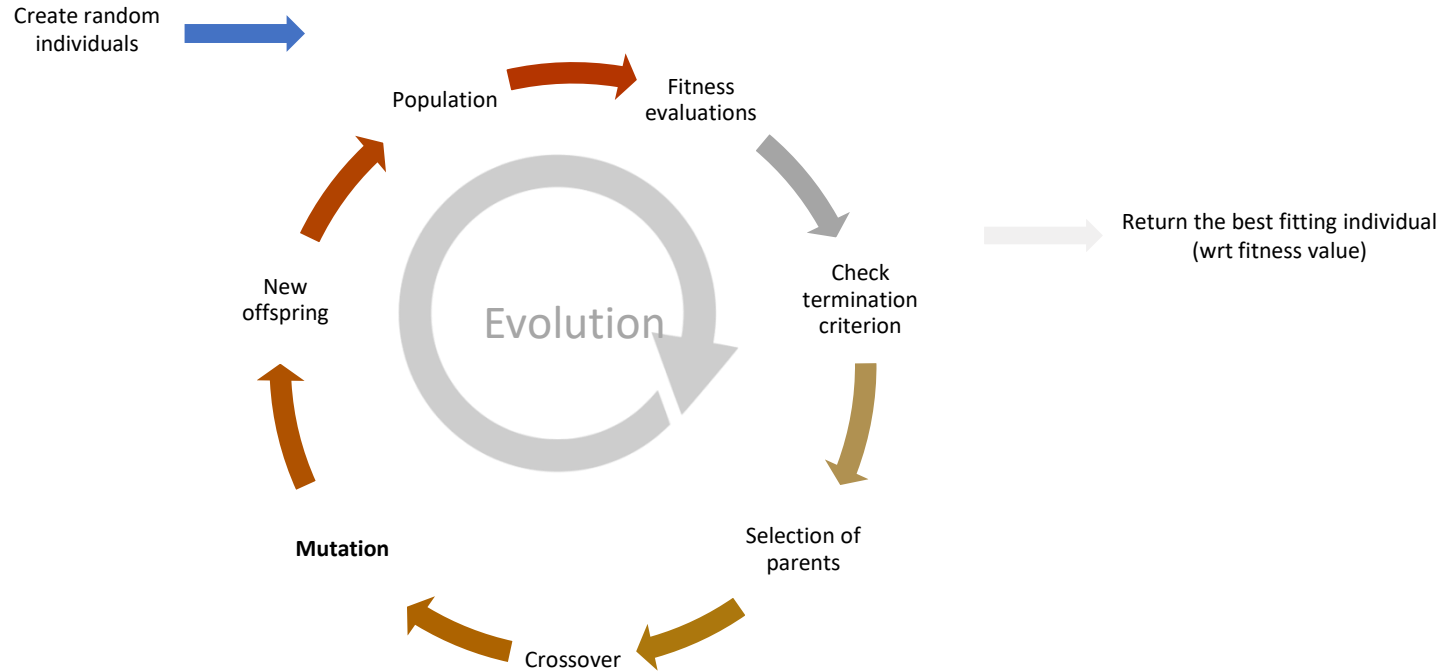


New mapping system: $4 \leftrightarrow 1$, $5 \leftrightarrow 8$, $6 \leftrightarrow 7$



- The remaining values **are changed using the mapping** defined at the beginning (e.g., 1 becomes 4 in the offspring)

GAs iterative process





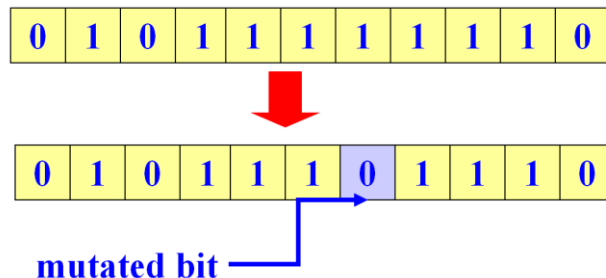
Mutation

- Remember: after selection, a part of the parents undergoes **crossover** (with probability P_c) and the remainder of the parents is left untouched (with probability $1 - P_c$)
- After crossovers are calculated, the **mutation operator is applied to all individuals**
 - Sometimes, with a probability P_m
- **Mutation introduces new genetic material** into the population, allowing big “jumps” inside the search space (i.e., escape local minima)
- There exist **many mutation operators**; the most common is the **uniform mutation**

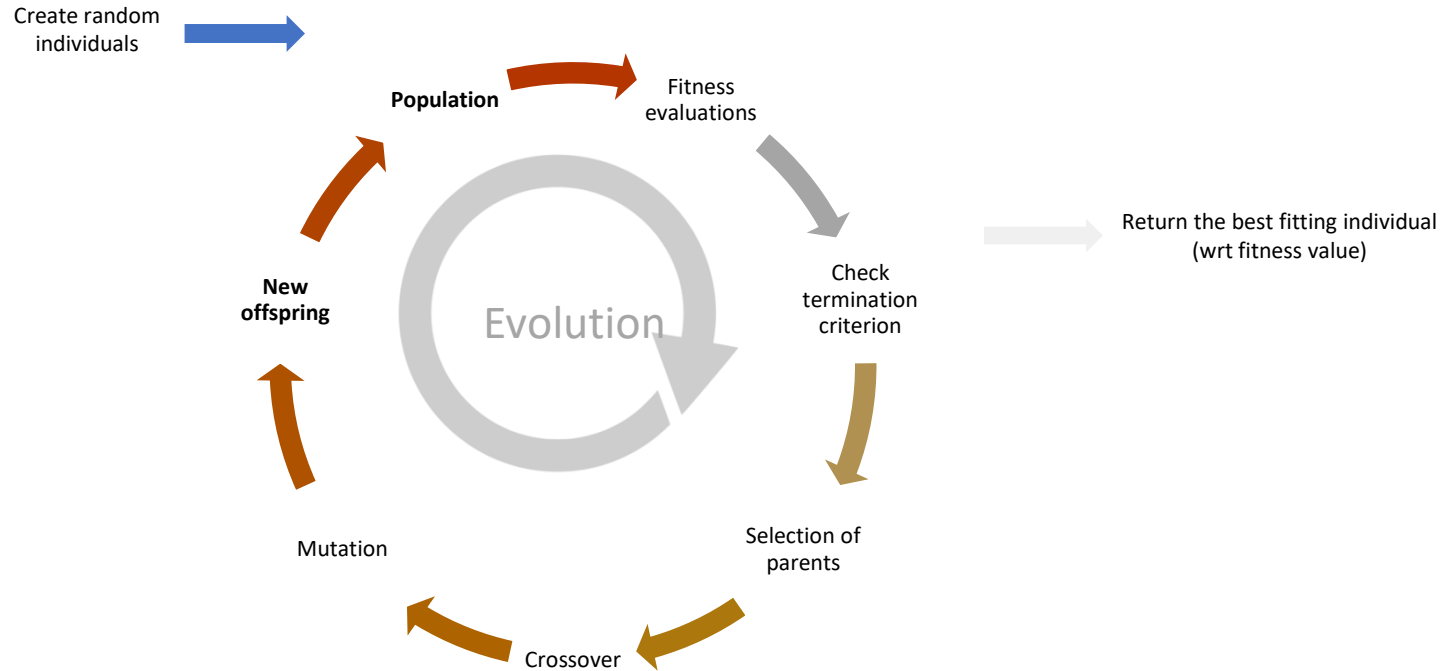
Uniform mutation

- In the case of candidate solutions encoded as binary strings (e.g., One Max) the mutation can be implemented as **bit flip**
- Mutation is applied to each bit of a solution with a **small probability** P_b
- Observation 1: a high mutation probability corresponds to a **random search!**
- Observation 2: $P_b \approx 0$ means that search is only driven by crossover: the GA cannot identify the global optimum if its “pieces” are not present in the initial population

Uniform mutation:



GAs iterative process



Elitism



- During the evolution, one **excellent individual** (with respect to the fitness) might be **“destroyed”** by the **genetic operators**
- Although it might “reappear” (think of **convergent evolution** in nature), the GA might take many generations to evolve it again



- **Elitism preserves such individual** by copying the best individual unaltered to the next generation (can also be applied to a group of individuals)

GA hyper-parameters (meta-optimization)



Name	Values	Notes
Crossover probability	$P_c \in [0,1]$, generally large (≈ 0.9)	Determines the probability of parents of undergoing crossover
Mutation probability	$P_b \in [0,1]$, generally small (≈ 0.05)	Controls the probability of random mutations to candidate solutions
Number of individuals	$N \in \mathbb{N}$, generally not too small (≈ 100)	A small population quickly loses diversity and cannot explore the search space effectively
* Tournament size	$k \in \mathbb{N}$, $2 \leq k \ll N$	Tournament size controls the selection pressure
* Iterations	$T_{max} \in \mathbb{N}$, generally large ($\approx 1000, 10000$)	The longer, the better

Observation 1: these settings have a **large impact on the optimization performance**

Observation 2: the optimal choice of these hyper-parameters is **problem-dependent**

Observation 3: the optimal initialization, selection, crossover, mutation, termination and elitism mechanisms are also **problem-dependent**

* These depend on the type of selection and termination mechanisms

Premature convergence and loss of diversity



Premature convergence: when a GA converges quickly to a **suboptimal population**

- Can be detected by analyzing the **change of the fitness values** of the population (e.g., the average fitness is not changing throughout generations)
- Observation: the stop of improvement can be a bad termination criterion!

Loss of diversity: when the individuals of a GA population are **too similar**, so that the crossover is no longer effective

- Can be detected by **comparing the structures** of candidate solutions (e.g., all individuals are similar or the new offspring are similar to the parents)

The two phenomena are the two faces of the **same coin**

- Are generally due to a poor selection of **hyper-parameters** (e.g., mutation insufficient to escape local minima, or too strong to preserve good schemata)

Comparing the performances

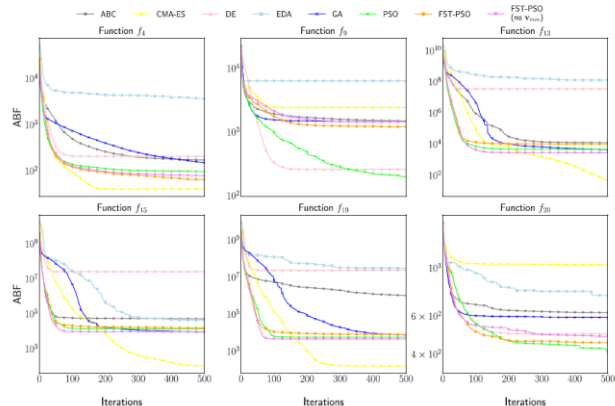


- The performance of a GA strongly depends on the used hyper-parameters
 - There are several ways to compare the performances of two GAs (e.g., hypothesis testing), including software tools (e.g., DSCtool [\[Eftimov et al., Applied Soft Computing 2020\]](#))
 - The simplest way to compare two or more GA (or algorithms) is **to run the same optimization multiple times** (say, 30 times) and **collect the fitness values** of the whole population during each iteration
 - By using this information, we can calculate and plot some statistics, e.g., the **average best fitness (ABF) at each iteration**: studying the curve we can assess the convergence speed, premature convergence, ecc.
- Alternatively, we can check the **distribution of fitness values** of the best solution found

Example of comparisons



A. Tangherloni, S. Spolaor, P. Cazaniga et al. / *Applied Soft Computing Journal* 81 (2019) 105494

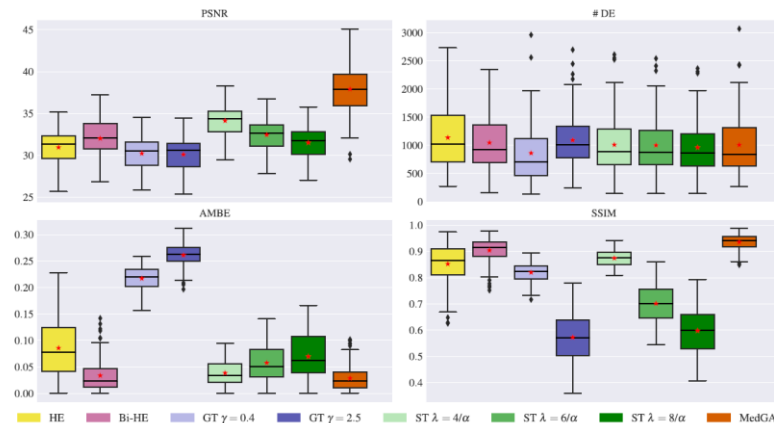


Each line corresponds to the ABF over 30 runs

Minimization problem: the lower, the better

Distribution of the fitness values of the best solutions

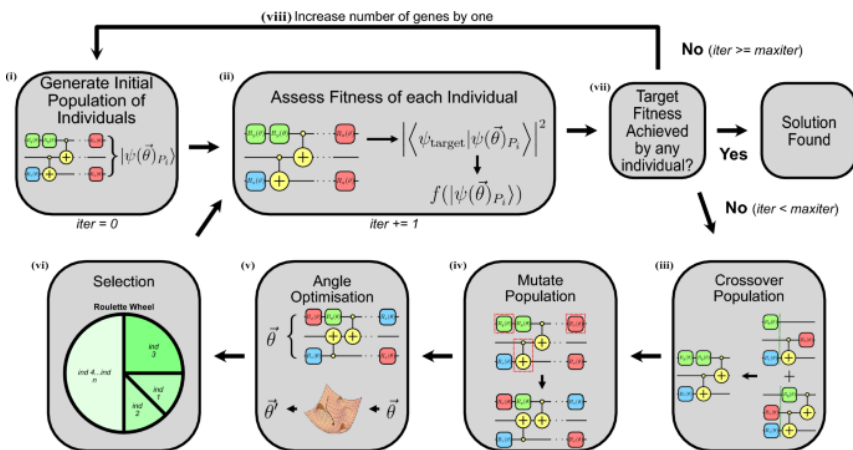
L. Rundo, A. Tangherloni and M.S. Nobile et al. / *Expert Systems With Applications* 119 (2019) 387–399



One example of real world application



- Goal: **low-depth state preparation** of quantum states in a quantum circuit
- One project last year was about a similar task!



Article | [Open access](#) | Published: 24 July 2023

GASP: a genetic algorithm for state preparation on quantum computers

Floyd M. Creevey, Charles D. Hill & Lloyd C. L. Hollenberg

Scientific Reports **13**, Article number: 11956 (2023) | [Cite this article](#)

7105 Accesses | **23** Citations | **1** Altmetric | [Metrics](#)

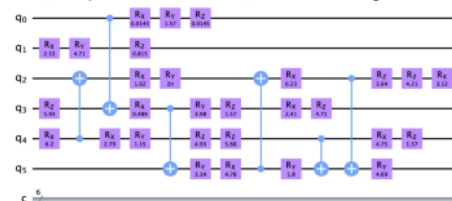
Abstract

The efficient preparation of quantum states is an important step in the execution of many quantum algorithms. In the noisy intermediate-scale quantum (NISQ) computing era, this is a significant challenge given quantum resources are scarce and typically only low-depth quantum circuits can be implemented on physical devices. We present a genetic algorithm for state preparation (GASP) which generates relatively low-depth quantum circuits for initialising a quantum computer in a specified quantum state. The method uses a basis set of R_x , R_y , R_z , and CNOT gates and a genetic algorithm to systematically generate circuits to synthesize the target state to the required fidelity. GASP can produce more efficient circuits of a given accuracy with lower depth and gate counts than other methods. This variability of the required accuracy facilitates overall higher accuracy on implementation, as error accumulation in high-depth circuits can be avoided. We directly compare the method to the state initialisation technique based on an exact synthesis technique by implemented in IBM Qiskit simulated with noise and implemented on physical IBM Quantum devices. Results achieved by GASP outperform Qiskit's exact general circuit synthesis method on a variety of states such as Gaussian states and W-states, and consistently show the method reduces the number of gates required for the quantum circuits to generate these quantum states to the required accuracy.

GA vs Qiskit

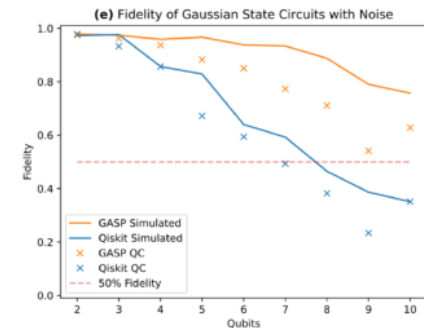
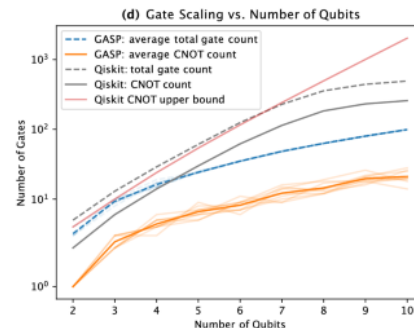
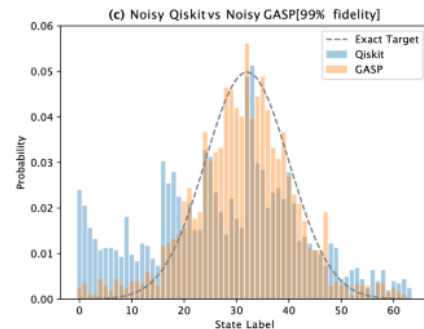
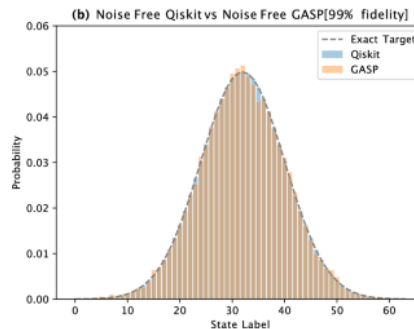
- GA outperforms Qiskit's det algo
 - Less gates** (13 vs 120)
 - In particular, less controlled NOT gates
 - More robust to noise!**
- [Creevey et al., Sci Reports 2023]

(a) Sample SolutionCircuits: n=6, Gaussian State Target



GASP: Depth = 13, Gates = 35

Qiskit (SBM): Depth = 120, Gates = 125



Queens!

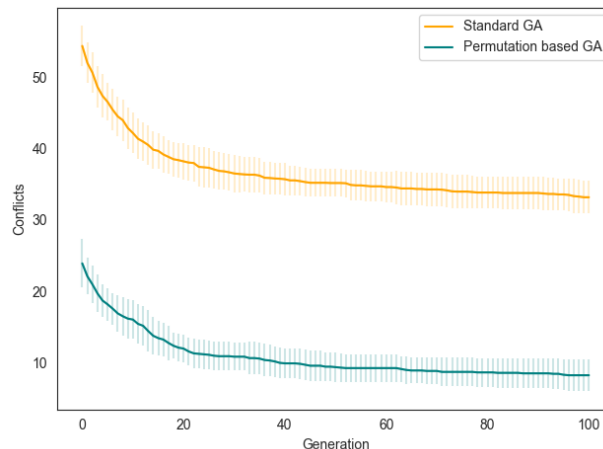
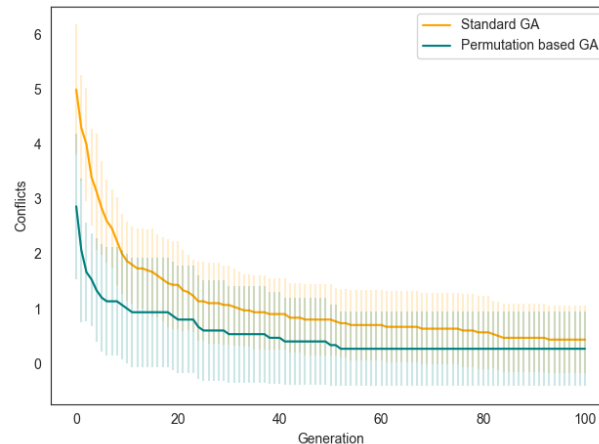
I implemented two GAs for the **Queen problem**

- N=8 (top) and N=30 (bottom)

Two alternative GAs

- **One naive version** (orange)
- **One based on permutations** using PMX crossover (teal)

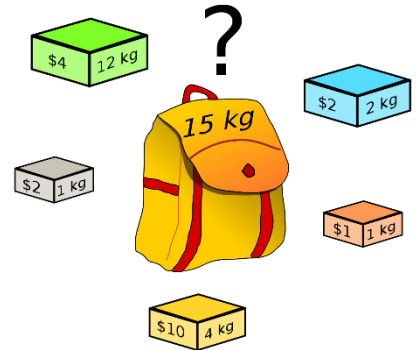
- What do you observe? :)



Handling constraints [1/2]



- It can be the case that individuals represent **unfeasible solutions** for a problem
- E.g., consider the **knapsack problem** (resource allocation)
 - Given a set of items, each with weight and value
 - Goal: determine which items include to that the total **weight** is lesser than or equal to the **knapsack limit** and the **overall value is maximized**
 - We can represent the items in the bag as **bits**
 - The GA *can* evolve individuals representing **unfeasible solutions** (i.e., overall weight is beyond the knapsack limit)



Handling constraints [2/2]



Explicit strategies	Implicit strategies
Set the fitness of unfeasible solutions to extreme values For instance, 0 or float_max The GA can become very inefficient when many solutions are unfeasible	Use special encodings The GA always explore feasible regions Might be very difficult (or even impossible) to implement for specific problems
Penalize the fitness function Smaller probability of being selected in future generations	Manipulate the search space A common GA can be exploited, the semantic of the solution changes before the fitness evaluations
"Fix" wrong solutions, e.g., random mutation Can be computationally very expensive, affect the exploration capability, create infinite loops, etc.	

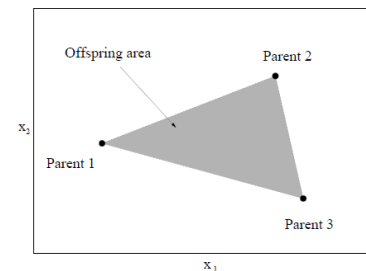
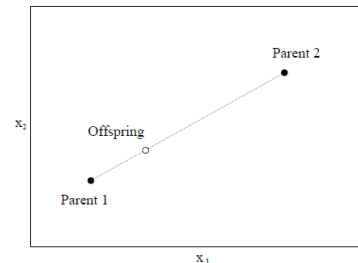
Real-coded GAs

- Most real-world problems concern the optimization of **vectors of real variables** but **binary encoding is not really efficient**
 - Simple example: the optimal weights of a neural network. What are solutions to this problem? How to encode GA candidate solutions for such a problem?
- Solution: **real coded GAs**
 - They **directly use real values** in chromosomes
 - In the neural network example, such real values represent they weights of the network
 - Each m -th component of the chromosome can take values in the interval $[v_m^{min}, v_m^{max}]$
 - Several **mutation** and **crossover operators** for real valued GAs exist

Example of real-valued crossover



- **Arithmetic crossover:** the offspring genome is generated by a (weighted) mean of each component in the two parent genomes:
- $w\mathbf{x}_1 + (1 - w)\mathbf{x}_2$
 - $w \in [0,1]$
 - If $w = 0.5$ then the offspring is the exact mean of the parents
 - w can be chosen randomly
 - A vector of weights $\mathbf{w}$ can be used (instead of a scalar)
 - **Multiple weighted individuals** can be used



Examples of real-valued mutation

- **Uniform mutation**: select a position m in the chromosome and change that value with a random number extracted with uniform distribution in $[v_m^{min}, v_m^{max}]$
- **Gaussian mutation**: each position m in the chromosome is changed by sampling a random value from a Gaussian distribution with $\mu = v_m$ and σ chosen by the user
 - Observation 1: Gaussian perturbations can create **invalid** candidate solutions (!)
 - Observation 2: the effectiveness of Gaussian mutation **depends on σ** : too low, the algorithm might not explore enough; too high, the algorithm cannot fine-tune the solutions
 - Observation 3: σ could be generation-dependent, or be performance-dependent, or have different settings for each dimension of the optimization problem...

GA in practice: DEAP



- **DEAP**: Distributed Evolutionary Algorithms in Python

- Framework for **evolutionary computation**
 - Designed for **rapid prototyping** and testing of ideas
 - Provides a lot of **already implemented pieces** to build your own genetic algorithm (and other evolutionary methods)
 - Check the **documentation** and the examples: <https://pypi.org/project/deap/>
-
- I used DEAP for the Queens, around 80 lines of code



DISTRIBUTED
EVOLUTIONARY
ALGORITHMS IN
PYTHON

Why using GAs

- Concept is easy to understand and extend
- The idea is general and decoupled from practical applications
- Supports multi-objective optimization (e.g., NSGA-II, Lecture 3)
- Effective with noisy problems
- Effective with non-differentiable problems
- Effective with non-analytic problems
- Inherently parallel algorithm (can leverage HPC)
- Always provides an optimal solution, which improves over time
- Can be integrated with other Computational Intelligence techniques like fuzzy logic, machine learning, and deep learning (e.g., [neuroevolution](#))

Applications of GAs

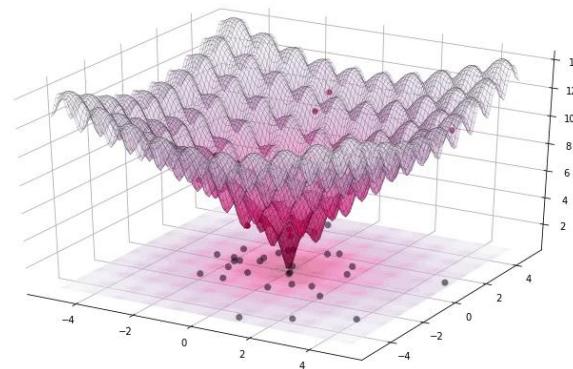


- **Optimization and design:** numerical optimization, circuit design, airplane design, factory scheduling, drug design, network optimization
- **Automatic programming:** evolving computer programs (e.g., Genetic Programming), evolving cellular automata
- **Machine Learning:** robot navigation, evolution of rule-based systems, neuro-evolution, adaptive security, adaptive interfaces
- **Scientific models of complex systems:** find the optimal parameters of models in economics, immunology, ecology, systems biology, synthetic biology, computational biology

Differential Evolution

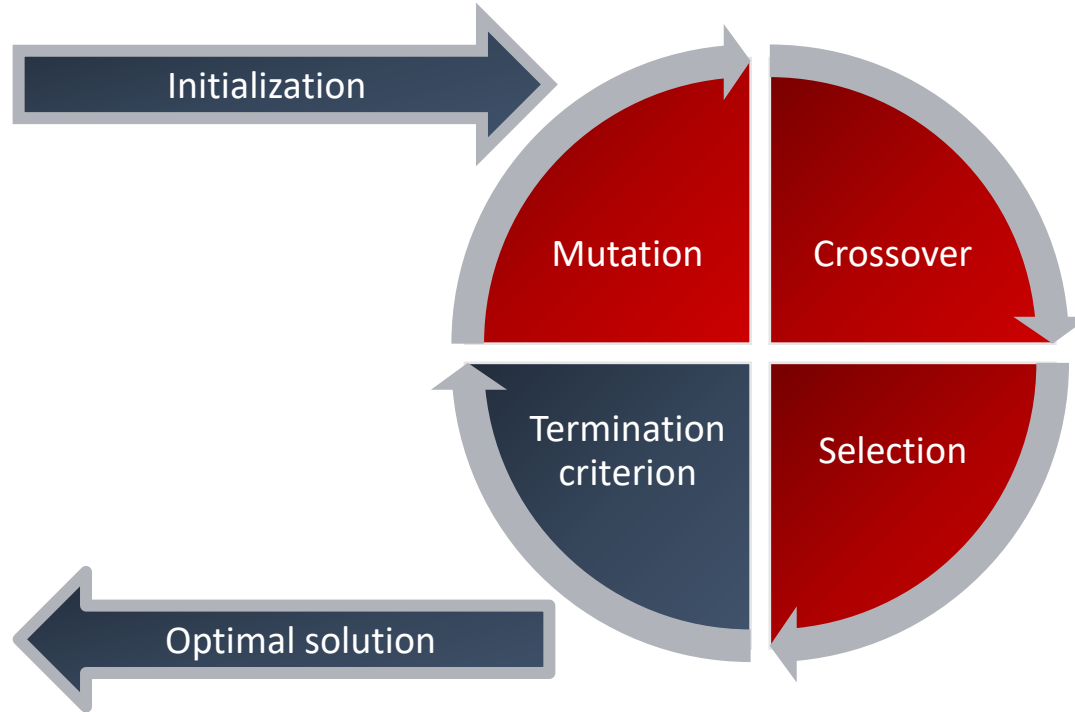


- **Differential Evolution (DE)** was introduced by Storn and Price in 1997
 - [Storn and Price, J Glob Optim 1997]
 - Defined «a parallel direct search method based on parameters vectors» for **real-valued GO**
 - **Evolutionary Computation (EC) approach**: a population of solutions «evolves» throughout the generations
 - Main difference wrt GAs: the scheme for the **generation of new individuals**, i.e., «the generation of new parameter vectors by adding a **weighted difference vector** – between two population members – to a **third member**»
 - Second difference wrt GAs is the **selection**: if the offspring vector yields a better objective function than the parent then the newly generated individual **replaces the parent**

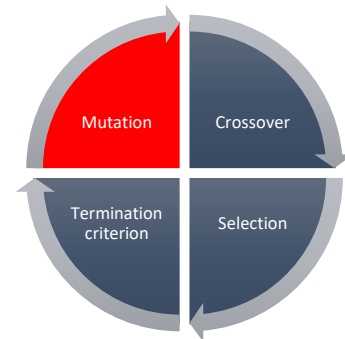


DE optimizing a 2D Ackley benchmark function
(figure by [Pablormier](#), CC BY-SA 4.0)

Differential Evolution in a nutshell



Differential Evolution – Mutation

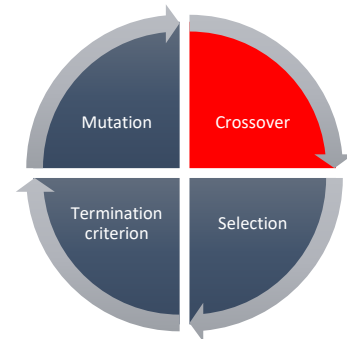


- Select one «designated» candidate solution $\mathbf{x}_i^G$
- Select **three** further random candidate solutions $\mathbf{a}, \mathbf{b}, \mathbf{c}$
- In the classic version of DE $\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{x}_i$ are **distinct**
- The so-called «**donor**» **vector** $\mathbf{v}_i$ is calculated as

$$\mathbf{v}_i = \mathbf{a} + F \cdot (\mathbf{b} - \mathbf{c})$$

- $F \in [0,2]$ is called the **mutation factor**
- The operation is also known as **differential mutation**

Differential Evolution – Crossover



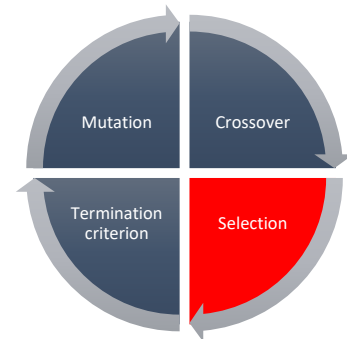
- The designated solution $\mathbf{x}_i^G$ and the donor vector $\mathbf{v}_i$ are **recombined** into a «trial» vector $\mathbf{u}_i$
 - The trial vector is mainly created using the elements of the designated solution $\mathbf{x}_i^G$
 - Elements of the donor vector $\mathbf{v}_i$ are selected for the trial vector with a **crossover probability** $CR \in [0,1]$

$$u_{i,d} = \begin{cases} v_{i,d} & \text{if } rnd_{i,d} < CR \vee I_{rnd} = d \\ x_{i,d}^G & \text{otherwise} \end{cases}$$

for all $d = 1, \dots, D$

- In the formula
 - $rnd_{i,d}$ is a random number sampled with uniform distribution in $[0,1]$
 - I_{rnd} is a random integer number chosen from $[1, \dots, D]$
 - This is sometimes called «binomial crossover» (see [Qin et al., IEEE Tran Evol Comp 2009])

Differential Evolution – Selection

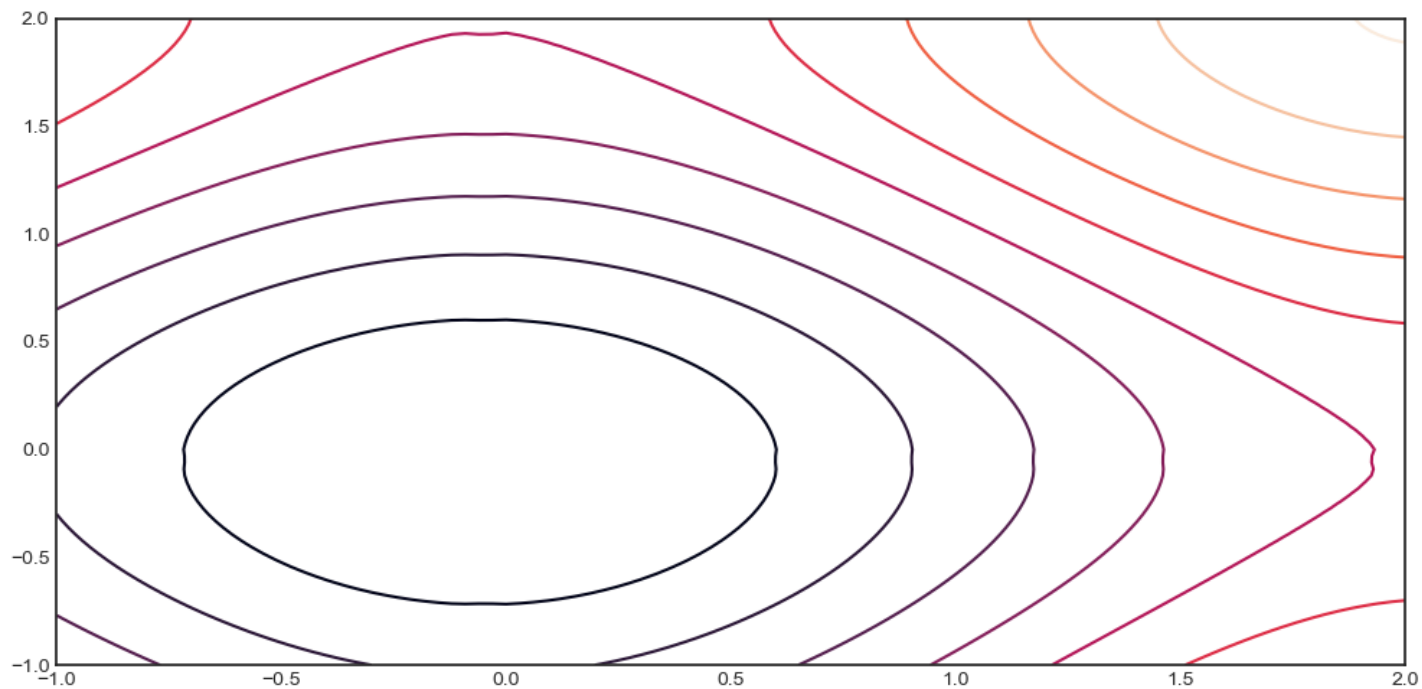


- The designated solution $\mathbf{x}_i$ is **compared** with the trial vector $\mathbf{u}_i$
 - **The best** of the two solutions (wrt the fitness function) is **selected**
 - E.g., in the case of minimization:

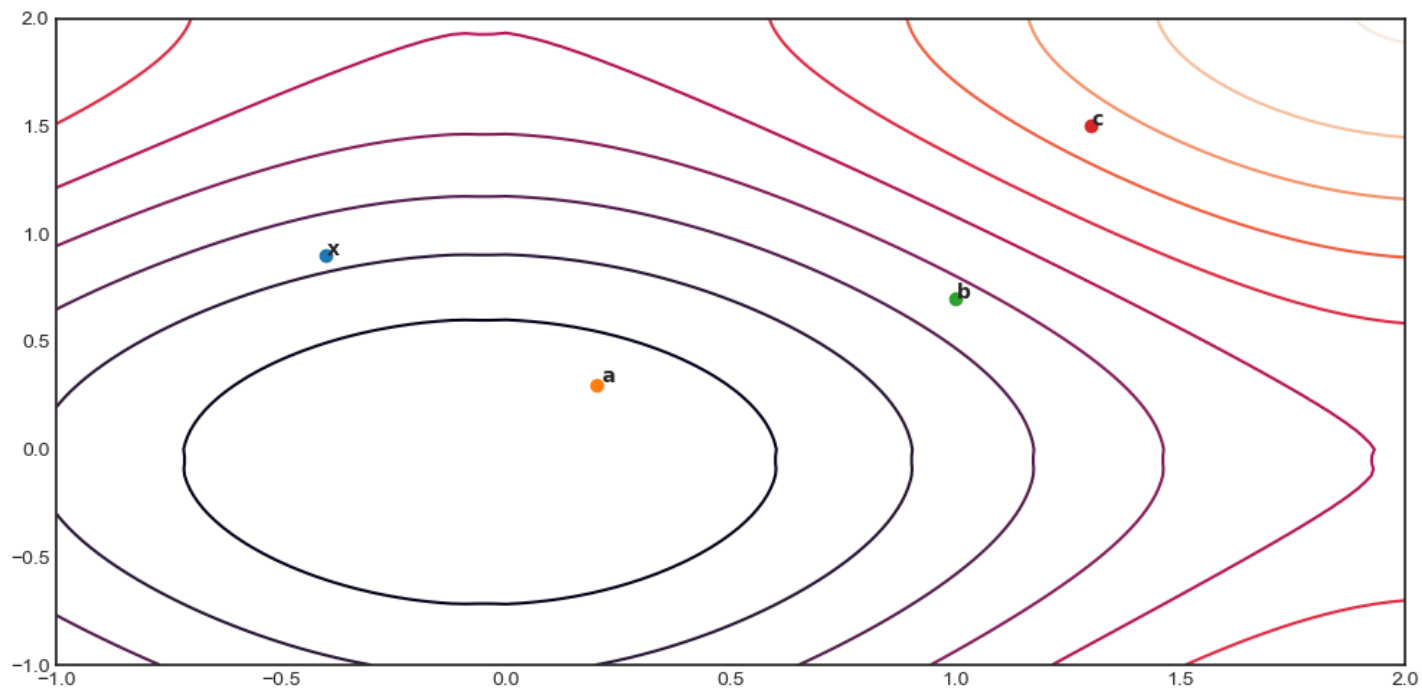
$$\mathbf{x}_i = \begin{cases} \mathbf{u}_i & \text{if } f(\mathbf{u}_i) < f(\mathbf{x}_i) \\ \mathbf{x}_i & \text{otherwise} \end{cases}$$

- The mutation-crossover-selection process is repeated for all $i = 1, \dots, N$ individuals in order to create a **new generation** of candidate solutions
- Generation after generation, DE converges to an optimal solution wrt to f and the algorithm halts when a **termination criterion** is met

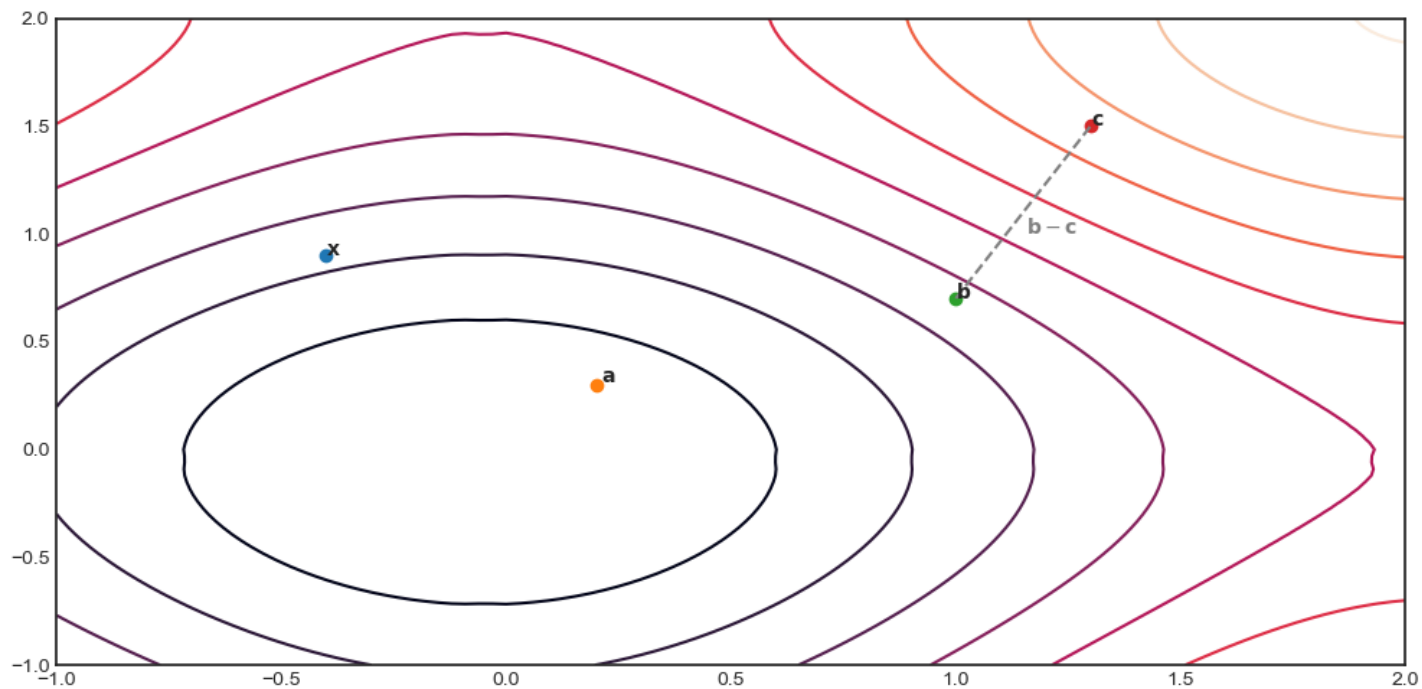
Example of DE



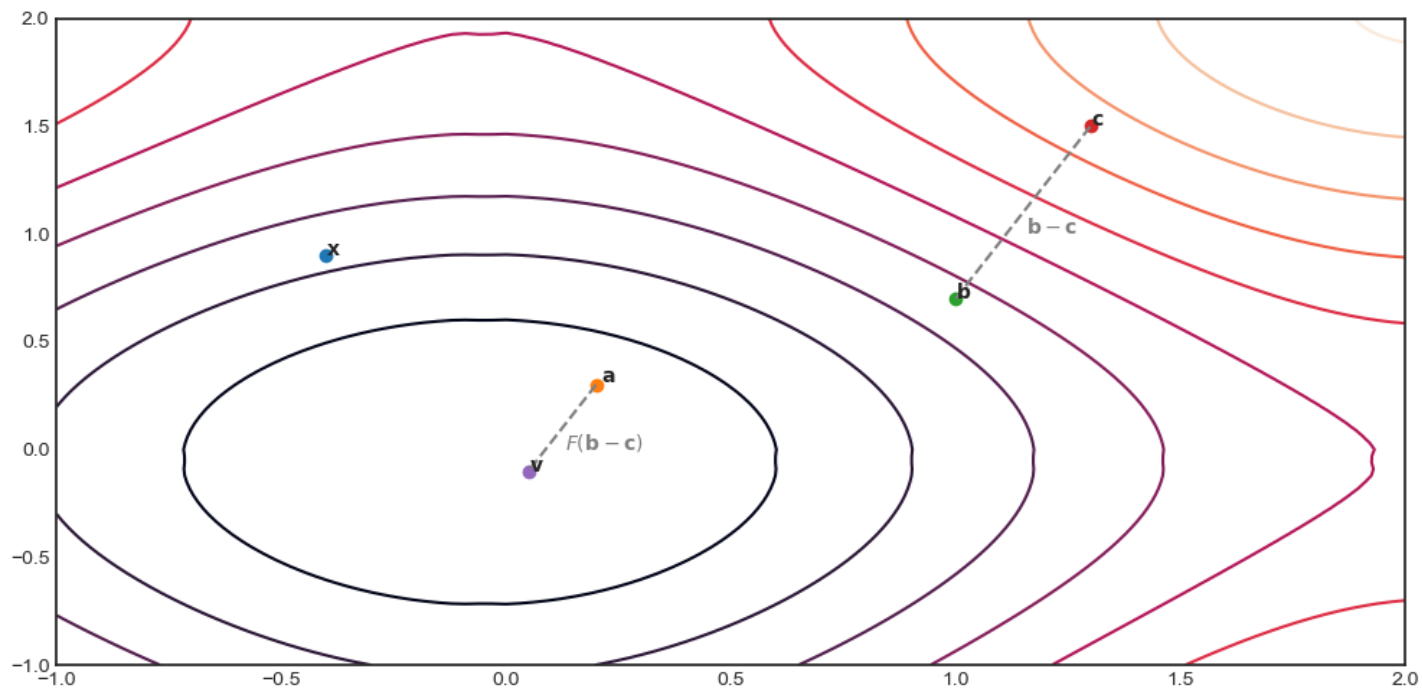
Example of DE



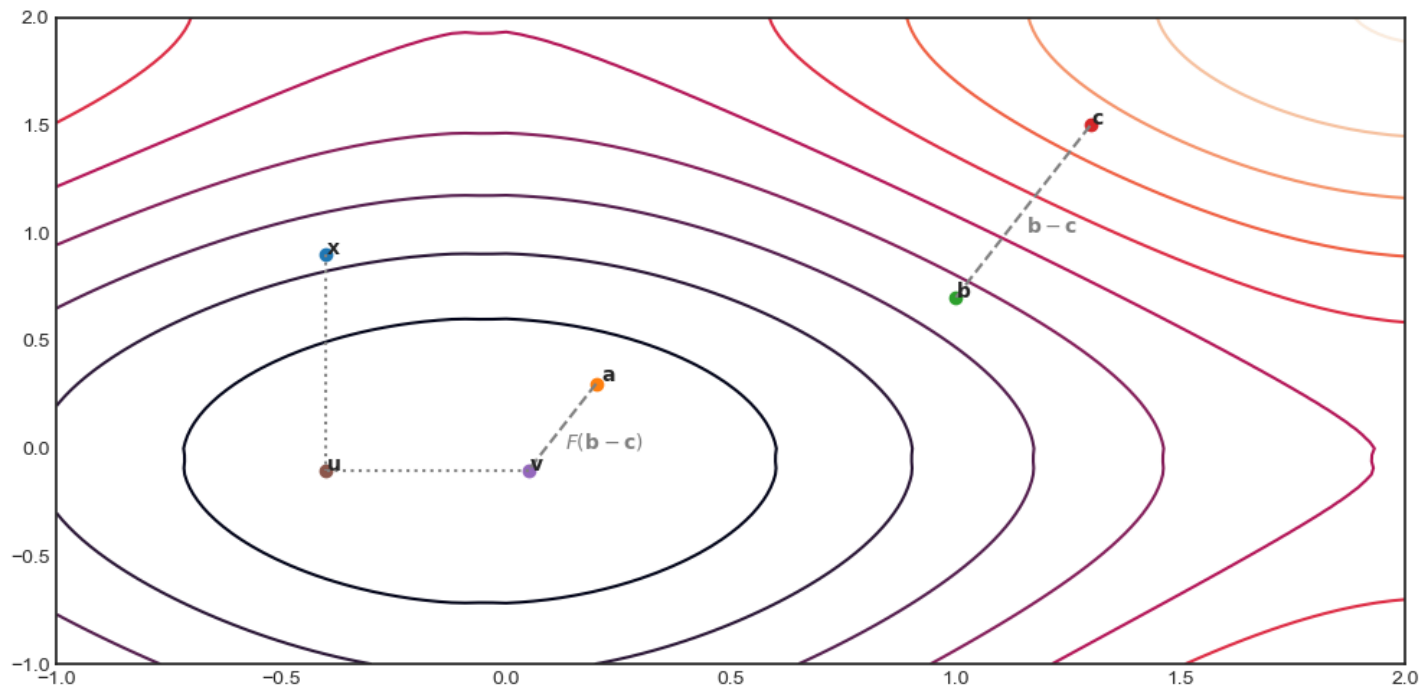
Example of DE



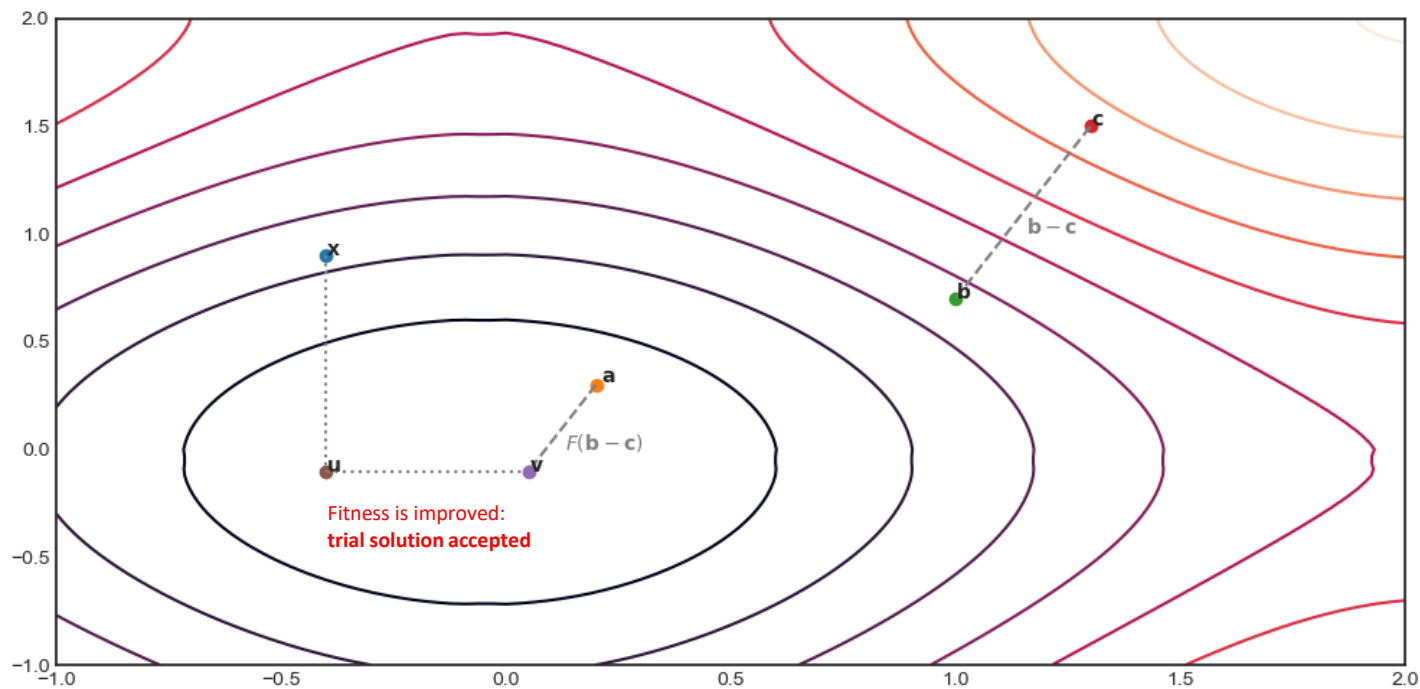
Example of DE



Example of DE



Example of DE



DE taxonomy

- The DE scheme just described is known as *DE/rand/1*
 - *DE* is self-explanatory
 - *rand* means that we pick a random individual as first vector for differential mutation
 - *1* means that we create the donor vector by using a single differential mutation
- There are some alternative schemes, most notably:

Name	Differential mutation equation
<i>DE/best/1</i>	$\mathbf{v}_i = \mathbf{x}_{best} + F \cdot (\mathbf{b} - \mathbf{c})$
<i>DE/current-to-best/1</i>	$\mathbf{v}_i = \mathbf{x}_i + F \cdot (\mathbf{x}_{best} - \mathbf{x}_i) + F \cdot (\mathbf{b} - \mathbf{c})$
<i>DE/rand/2</i>	$\mathbf{v}_i = \mathbf{a} + F \cdot (\mathbf{b} - \mathbf{c}) + F \cdot (\mathbf{d} - \mathbf{e})$
<i>DE/rand-to-best/1</i>	$\mathbf{v}_i = \mathbf{x}_i + F \cdot (\mathbf{x}_{best} - \mathbf{x}_i) + F \cdot (\mathbf{b} - \mathbf{c})$

$\mathbf{x}_{best}$ is the best candidate solution found so far
 $\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}, \mathbf{e}$ are randomly selected individuals

Why using DE?



- There exist competitions of global optimization (e.g., BBOB, CEC)
- **Advanced variants of DE** (e.g., DISH, L-SHADE) seem to **outperform competitor algorithms** (at least, on benchmark functions)
- In general, it is considered one of the **state-of-the-art optimization algorithms**

Summary



- Evolutionary Computation: the idea of using Darwinian processes to evolve a (random) population of candidate solutions towards the global optimum (with respect to a fitness function)
- Evolution is effective for both discrete (e.g., genetic algorithms) and real-valued (e.g., differential evolution) optimization problems

The schema theorem



The schema theorem is **one of the strongest theoretical results** about GAs

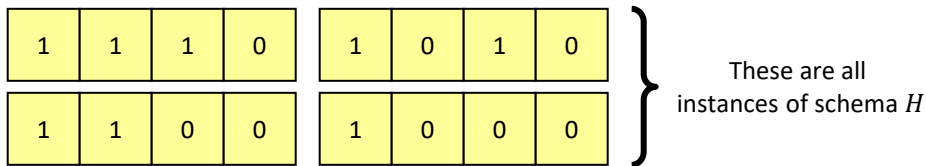
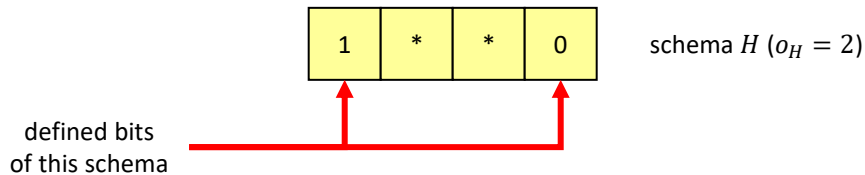
- **Fundamental theorem of GAs**
- **Asymptotic convergence proof** by J. Holland [Holland, *Adaptation in Natural and Artificial Systems*, The MIT press 1975]
- Intuitive short explanation: **high quality “parts”** (the schemata) that are contained in candidate solutions **increase exponentially** in the population in successive generations
- Assume maximization throughout the proof



What is a schema

A **schema** H is a bit string with 1s, 0s and don't care symbols *

- It is a “template” representing parts of a chromosome
- **Order of schema:** number of defined bits in the schema (i.e., different from *)
- The order of the schema H is denoted by o_H



Further definitions about schemata

- The **length of the schema** l_d is the distance from the left- and right-most positions that are different from *

*	*	1	0	*	0	*	0
---	---	---	---	---	---	---	---

$$M = 8, l_d = 5, o = 4$$

0 1 2 3 4 5 6 7

- The **expected number of individuals belonging to schema H** at generation t is denoted by $m_H(t)$
 - $m_H(0)$ is the number of individuals with schema H in the initial population
- The **average fitness** of the population will be denoted by $\hat{f} = \frac{1}{N} \sum_{n=1}^N f_n$

Schema theorem: the effect of selection

- Effect of N selections (assuming **roulette wheel**)

$$m_H(t+1) = m_H(t) \cdot N \cdot \frac{f(H)}{\sum_{n=1}^N f_n}$$

The probability of schema H to survive depends on its fitness and numerosity

- Observe that $N / \sum_{n=1}^N f_n = \hat{f}^{-1}$ so that we can rewrite the formula as

$$m_H(t+1) = m_H(t) \cdot \frac{f(H)}{\hat{f}}$$

If $f(H) > \hat{f}$ then the schema H will increase in the next generation

- We can see that individuals with **higher fitness than the average** of the population have an **evolutionary advantage**. Implications?

Schema theorem: again on selection

- Assume that $f(H) = \hat{f}(1 + c)$ for some (small) value $c > 0$

- Replace in $m_H(t + 1) = m_H(t) \cdot \frac{f(H)}{\hat{f}}$ to obtain:

$$m_H(t + 1) = m_H(t) \cdot \frac{\hat{f}(1 + c)}{\hat{f}} = m_H(t) \cdot (1 + c)$$

- We can prove by induction that: $m_H(t) = m_H(0) \cdot (1 + c)^t$
- Hence, individuals belonging to schema H actually **increase exponentially**

Schema theorem: effect of crossover

- Long schemata have a higher probability of being “broken” by crossover, i.e., the **survival** of the schema is related to **its length**:

$$p_s(H) = 1 - \frac{l_d}{M - 1}$$

- If we assume crossover probability $P_c < 1$ and the possibility that the genetic material coming from the other parent can “fix” (by chance) the broken schema we obtain:

$$p_s(H) \geq 1 - P_c \frac{l_d}{M - 1}$$

- We can now combine the effect of selection to the effect of crossover. Observe that crossover is not generation-dependent: the **probability remains exponential**:

$$m_H(t + 1) = m_H(t) \cdot \frac{f(H)}{\hat{f}} \cdot \left[1 - P_c \frac{l_d}{M - 1} \right]$$

Schema theorem: effect of mutation



- Consider mutation as bit flip: the probability of a bit in the schema to survive depends on the **mutation probability** and it is equal to $1 - P_b$
- For the whole schema, **all bits different from *** must survive too, that is: $(1 - P_b)^{o_H}$
- This leads to the final formula: $m_H(t + 1) = m_H(t) \cdot \frac{f(H)}{\hat{f}} \cdot \left[1 - P_c \frac{l_d}{M-1} \right] \cdot (1 - P_b)^{o_H}$
- Hence, the **probability of survival is higher for low-order schemata**

Schema theorem



- The theorem proves that individuals characterized by schema H increase exponentially if:
 - They have a **fitness better than the average**
 - The schema is **“short”**
 - The schema has **low order**
- Known as the “building blocks” theory
- We are only considering the “destructive” effects of genetic operators (they can also introduce good schemata into the population!)
- The exponential increment of building blocks implies an **asymptotic convergence** (i.e., taking the limit of $t \rightarrow \infty$) **to the global optimum**